

LLM Agent Performance Analysis: Trajectories and Outcomes

Short report for mini-SWE-agent-v2 trajectories of the top five SWE-bench models.

Method note. The required tool extracts message counts only: total messages and counts by system, user, assistant, and tool role. I additionally joined each trajectory with its resolved outcome. This label is not part of Task #1; it is used only to test and justify interpretations of the raw-count patterns.

1. Limitations of Raw Message Counts

Message counts are a cheap proxy for agent effort, but they do not directly measure reasoning quality. A long trajectory may be productive search, or it may be repeated failed edits, redundant tool calls, or recovery from an early wrong assumption. Therefore, raw counts should be read as activity volume and cost, not as proof of better reasoning.

2. Findings from Counts Only - Without Labels

Based on: histograms of raw message-count distributions.

- Fixed interaction structure: system and user messages almost always peak at 1, while assistant and tool messages are nearly symmetric. This suggests stable initialization and strict assistant-tool alternation.
- Fast-success hypothesis: most distributions peak at relatively low message counts. Since these are top leaderboard models with high overall success rates, many successful runs are plausibly concentrated in those early peaks. When a strong agent understands the task and picks a good repair path, it often converges quickly. The long tail is more suspicious: it usually means the agent is still searching, retrying, or drifting instead of closing on a fix.
- Different operational tempos: Claude 4.5 Opus, Claude Opus 4.6, and GPT5-2-Codex behave like lower-volume agents, with many runs around 20-60 messages. Gemini 3 Flash and MiniMax M2.5 use broader trajectories, often around 80-120 messages.
- Different stopping behavior: MiniMax has the longest tail and can exceed 500 messages, while GPT5-2-Codex and Claude models usually stop much earlier. This points to different tolerance for continuing after progress becomes unlikely.

3. Findings After Adding Resolved Labels

Based on: grouped histograms split by resolved outcome labels.

- Short runs are usually successful: the left-side peaks are mostly resolved tasks, supporting the idea that successful fixes are found inside an early working window.
- Long trajectories are mostly failures: very long runs are dominated by unresolved tasks. They usually reflect ineffective exploration, repeated failed patches, or iteration/token limits, not deep reasoning that eventually pays off.
- Efficiency matters: Claude 4.5 Opus achieves the best combination of high resolve rate and low message cost. Gemini 3 Flash and MiniMax M2.5 reach similar success levels, but spend many more messages per run, increasing latency and tool cost.
- Failure modes differ: GPT5-2-Codex often fails early, suggesting premature stopping or weak recovery. MiniMax often fails late, suggesting over-persistence in hopeless runs. These are opposite problems: one needs better persistence, the other better stopping criteria.

4. Consequences

Message count becomes useful only when paired with outcome. More messages do not automatically mean more capability; they may simply mean higher latency and tool cost. For deployment, the best agent reaches correct fixes with a low, predictable interaction budget. Once a trajectory passes the normal success window without clear progress, recovery probability drops, so the run should be stopped, restarted, or routed to another strategy.

Conclusion

Raw counts reveal agent style, but not success by themselves. With resolved labels, the main pattern is clear: successful runs are usually short, while very long trajectories are usually failed search loops. The strongest signal is efficiency under success: how often a model resolves tasks before entering the long tail. Thus, trajectory length is useful for comparing cost, latency, stopping behavior, and failure modes, but only together with outcome labels.